CoGes — Architecture technique V1.1

Version révisée de l'architecture logique et technique. Les décisions structurantes précédentes sont conservées — monolithe modulaire FastAPI, PostgreSQL, Next.js, Redis/Celery, outbox transactionnelle, isolation par `company_id`. Cette révision intègre le déploiement Vercel, sépare l'audit critique de l'audit asynchrone, précise l'implémentation réelle de RLS, réduit le périmètre V1 de Sales et Finance, et fait apparaître partout la frontière **V1 / versions futures**. La **V1.1** aligne ce document sur le **Modèle de données V1.1** du 21/09/2026, qui fait autorité en cas de divergence : nommage pointé des événements, responsabilité de `products`, cycle `draft → confirmed`, clés étrangères composites tenant.

PRINCIPE DIRECTEUR

Construire peu de fonctionnalités, mais les rendre fiables, cohérentes, traçables et exploitables par CoGes Intelligence. CoGes V1 n'est pas un ERP : toute fonctionnalité qui n'est pas nécessaire au cycle vente → stock → argent → décision est reportée.

CHEMIN UTILISATEUR	TRAITEMENTS LONGS	MODULES V1	AUDIT
Vercel · Next.js + API	**Docker hors Vercel**	**14 core + 5 gestion**	**Critique = transactionnel**

Ce que cette révision change

SUJET	AVANT	DÉCISION V1.1
Déploiement	Une image Docker API + une image worker	Chemin utilisateur sur Vercel (Next.js + API) ; base, cache, workers et ML hors Vercel, packagés en Docker
Audit Log	Alimenté par le bus d'événements	Deux canaux : audit critique dans la transaction métier, audit secondaire par événement
RLS	« RLS adossée à une variable de session »	Procédure d'injection, de transmission et de nettoyage du contexte sur pool documentée + tests de fuite obligatoires
Sales	Devis, commandes, factures, avoirs	Vente, lignes, remise, taxes, paiement total/partiel, annulation, retour, reçu simple
Finance	Périmètre implicite	Gestion, pas comptabilité générale : exclusions écrites
Intelligence	« Produits à forte valeur »	Huit métriques distinctes ; toute recommandation nomme sa métrique
Événements métier	SCREAMING_SNAKE_CASE (`SALE_CREATED`, `stock.updated`…)	Notation pointée `namespace.action` du modèle de données (`sale.confirmed`, `stock.updated`, `sale_return.refunded`) comme convention unique de `business_events.event_type`
Products	La fiche porte « prix de vente, coût »	Référentiel fonctionnel de l'article seul ; prix historisé par intervalles dans `product_prices`, stock projeté depuis `inventory_movements`
Cycle de vie d'une vente	Une seule étape de validation	Cycle en deux temps `draft → confirmed` : seule la confirmation déclenche stock, écriture financière, audit critique et outbox
Isolation tenant	Clé étrangère simple vers `companies`	Cible `UNIQUE (company_id, id)`, FK composites `(company_id, child_id) → (company_id, parent_id)` sur chaque relation tenant-scoped, RLS `FORCE`
Périmètre	Évolutions futures en fin de document	Marqueur V1 / Futur sur les modules, les tableaux et les diagrammes

DIAGRAMME 1 — ARCHITECTURE LOGIQUE GLOBALE

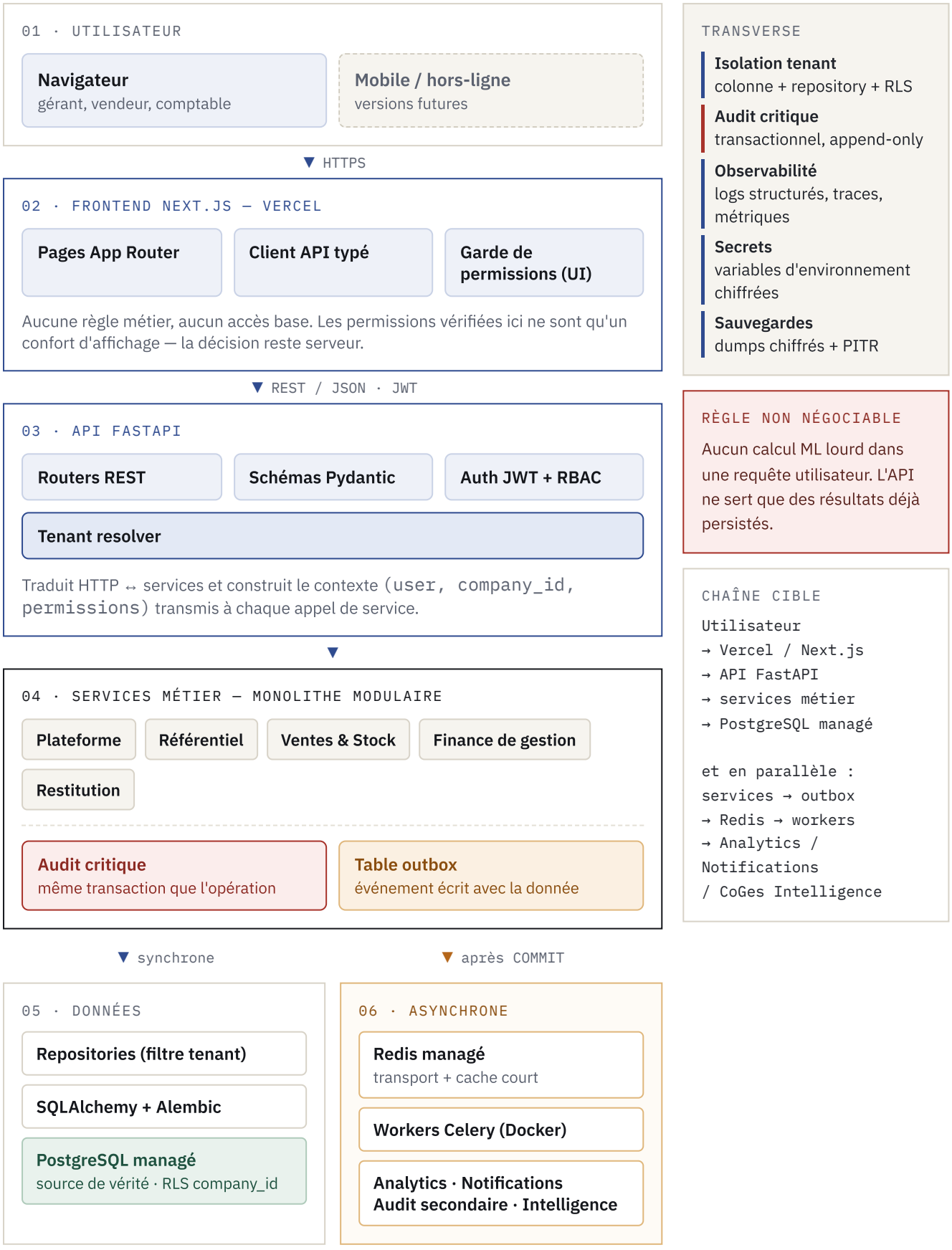


Figure 1 — Architecture logique. Le frontend n'atteint jamais PostgreSQL. L'audit critique est dans la transaction ; tout le reste part de l'outbox après commit.

Rôle de chaque couche

COUCHE	RESPONSABILITÉ	NE FAIT JAMAIS
Frontend Next.js	Rendu, navigation, validation de surface, affichage des résultats. Consomme uniquement l'API REST.	Règle métier, calcul de marge, accès base
API / routers	Contrat HTTP, (dé)sérialisation, authentification, résolution du tenant, codes d'erreur, pagination.	Requête SQL, orchestration métier
Services métier	Invariants, cas d'usage, frontière transactionnelle, audit critique, écriture outbox.	Connaître HTTP ou le SQL détaillé
Repositories	Requêtes, filtre systématique par `company_id`, mapping ORM.	Décider d'une règle de gestion
PostgreSQL	Vérité transactionnelle, contraintes, RLS, vues d'agrégation.	Porter de la logique applicative
Workers (Docker)	Agrégats, notifications, exports, features, entraînement et inférence ML.	Bloquer une requête utilisateur
CoGes Intelligence	Lecture analytique, features, modèles versionnés, prévisions, recommandations explicables.	Écrire dans les tables transactionnelles

DIAGRAMME 2 — DÉPLOIEMENT : VERCEL + INFRASTRUCTURE EXTERNE

Vercel n'est pas un remplacement de Docker. Vercel est la plateforme du chemin utilisateur : requêtes courtes, déploiement continu depuis Git, HTTPS et domaine gérés. Docker est l'environnement d'exécution de tout ce qui doit vivre longtemps ou calculer longtemps. La frontière est la durée de vie du processus, pas la technologie.

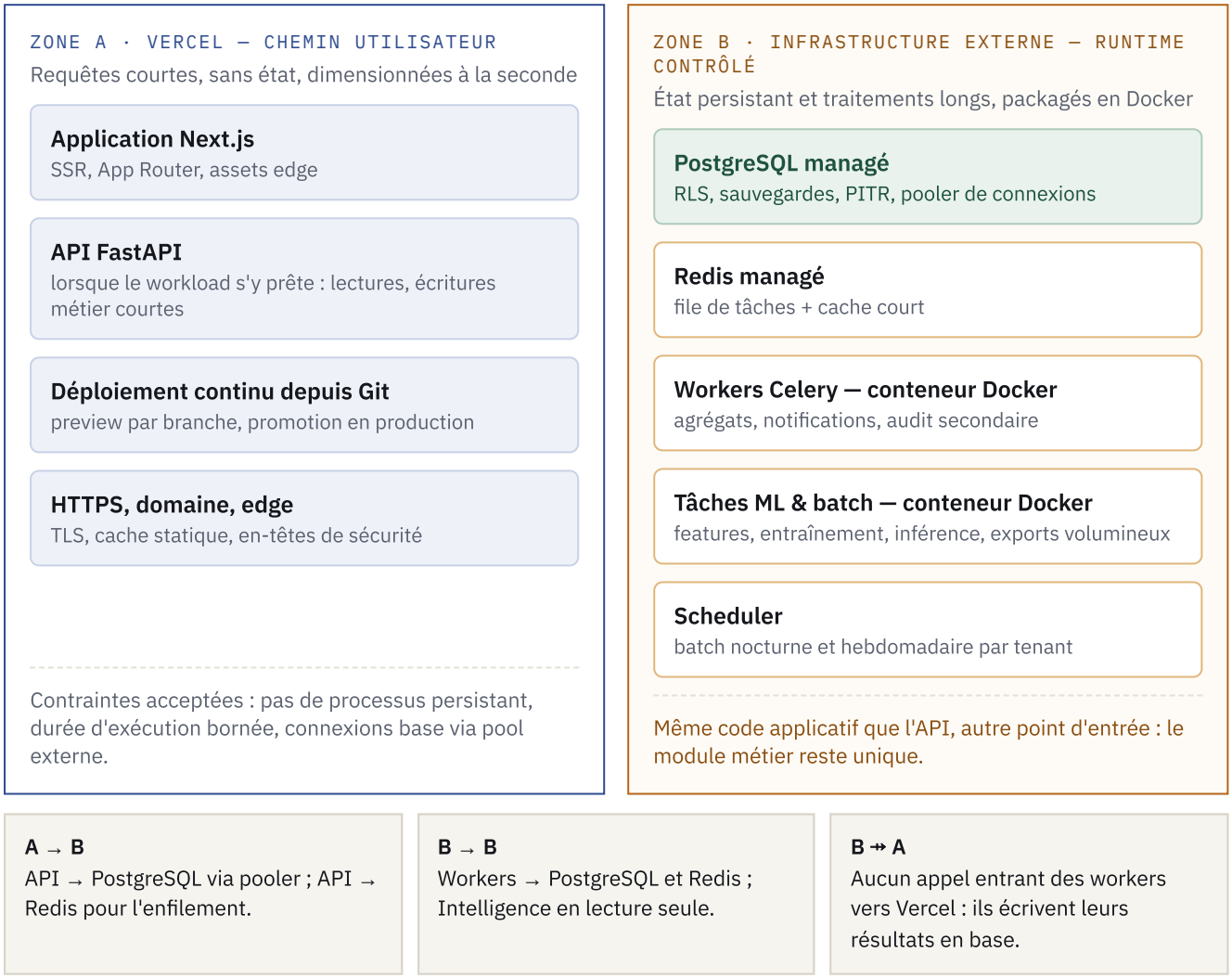


Figure 2 — Déploiement. Un composant relève de la zone B dès qu'il est persistant, long, planifié, ou consommateur de mémoire.

Règle d'affectation d'un composant

COMPOSANT / TRAITEMENT	ZONE	POURQUOI
Pages et rendu Next.js	**Vercel**	Chemin utilisateur, sans état, bénéficie du déploiement Git et de l'edge
Endpoints d'écriture métier (vente, paiement, stock)	**Vercel**	Transaction courte, cible < 300 ms
Endpoints de lecture et de tableau de bord	**Vercel**	Lisent des agrégats déjà calculés
Base de données	**Externe managé**	Persistance, sauvegardes, RLS, pooling
File de tâches et cache	**Externe managé**	État partagé entre API et workers
Workers Celery	**Docker externe**	Processus persistant qui consomme une file
Features, entraînement, inférence	**Docker externe**	Durée et mémoire incompatibles avec une requête HTTP
Exports volumineux, réconciliation nocturne	**Docker externe**	Tâches planifiées, fichier livré en fin de traitement
Migrations Alembic	**Docker externe**	Exécutées comme étape de déploiement, jamais au démarrage d'une requête

DIAGRAMME 3 — MODULES BACKEND

Cinq bandes, dépendances orientées vers le bas uniquement ; toute remontée d'information passe par un événement. Les modules marqués `V1` sont obligatoires ; ceux marqués `FUTUR` ne sont pas développés en V1 mais leur emplacement est réservé.



Figure 3 — Modules. Chaque module expose une façade de services et garde ses modèles privés ; un module n'importe jamais le repository d'un autre.

Responsabilités et dépendances

MODULE	STATUT	RESPONSABILITÉ V1	DÉPEND DE
Companies	V1	Entreprise, paramètres (devise, TVA activée ou non, seuils), racine d'isolation.	—
Authentication	V1	Login, refresh rotatif, hachage Argon2, révocation de session.	Users, Companies
Users	V1	Comptes, rattachement à une entreprise, cycle de vie.	Companies, Roles
Roles & Perms	V1	Rôles, matrice de permissions, vérification RBAC serveur. Tout changement est audité en transactionnel.	Companies, Audit
Products	V1	Fiche produit, unité, seuil d'alerte, taux de taxe par défaut ; prix de vente et coût historisés dans `product_prices`, jamais stockés sur la fiche. Stock = projection de `inventory_movements`.	Categories, Suppliers
Categories	V1	Classification du catalogue.	—
Customers	V1	Clients, coordonnées, encours autorisé, historique d'achat.	Companies
Suppliers	V1	Fournisseurs, délai de réapprovisionnement indicatif.	Companies
Sales	V1	Vente et lignes, prix appliqué, remise, taxes si activées, paiement total ou partiel, annulation, retour, reçu simple. Émet `sale.confirmed`, `sale.cancelled`, `sale.returned`, `sale_return.refunded`.	Products, Customers, Inventory, Finance, Audit
Inventory	V1	Mouvements (entrée, sortie, ajustement, retour), stock courant projeté, seuils. Émet `stock.updated`.	Products, Suppliers
Finance	V1	Revenus de ventes, charges, trésorerie simple, marge estimée, indicateurs de gestion.	Sales, Expenses, Payments, Debts
Payments	V1	Encaissements et décaissements, allocation à une vente ou une dette, idempotence. Émet `payment.received`.	Sales, Debts, Finance, Audit
Debts	V1	Créances clients et dettes fournisseurs, échéance, solde restant. Émet `debt.created`, `debt.settled`.	Customers, Suppliers, Finance, Audit
Expenses	V1	Dépenses, charges récurrentes, catégorisation. Émet `expense.created`.	Finance

MODULE	STATUT	RESPONSABILITÉ V1	DÉPEND DE
Dashboard	**V1**	Composition des KPI d'accueil sur agrégats pré-calculés.	Analytics, Notifications, Intelligence
Analytics	**V1**	Agrégats essentiels : CA, marge, top produits par métrique, séries temporelles, vues matérialisées.	lecture seule : Sales, Inventory, Finance
Notifications	**V1**	Alertes principales (stock bas, dette échue, anomalie), in-app et e-mail, déduplication.	Bus d'événements, Users
Audit Log	**V1**	Journal append-only à deux canaux : critique (transactionnel) et secondaire (événementiel).	Services métier (direct) + bus
Import/Export	**V1**	CSV/Excel du référentiel et des ventes, uniquement si nécessaire au démarrage. Export volumineux en worker.	Référentiel, Sales, workers
CoGes Intelligence	**V1 simple**	Statistiques descriptives, tendances, anomalies simples, alertes de stock, risque de rupture, prévisions simples, produits identifiés par métrique, recommandations explicables.	lecture seule : Analytics + tables de features

DIAGRAMMES 4 À 6 — FLUX MÉTIER

Chaque flux distingue la **partie transactionnelle** — une seule transaction PostgreSQL, atomique, dans la requête — de la **partie asynchrone** déclenchée après commit. L'audit critique et l'écriture dans l'outbox font partie de la transaction : aucun événement perdu, aucune notification pour une opération annulée par rollback, aucune trace d'audit dépendante de la disponibilité de Redis.

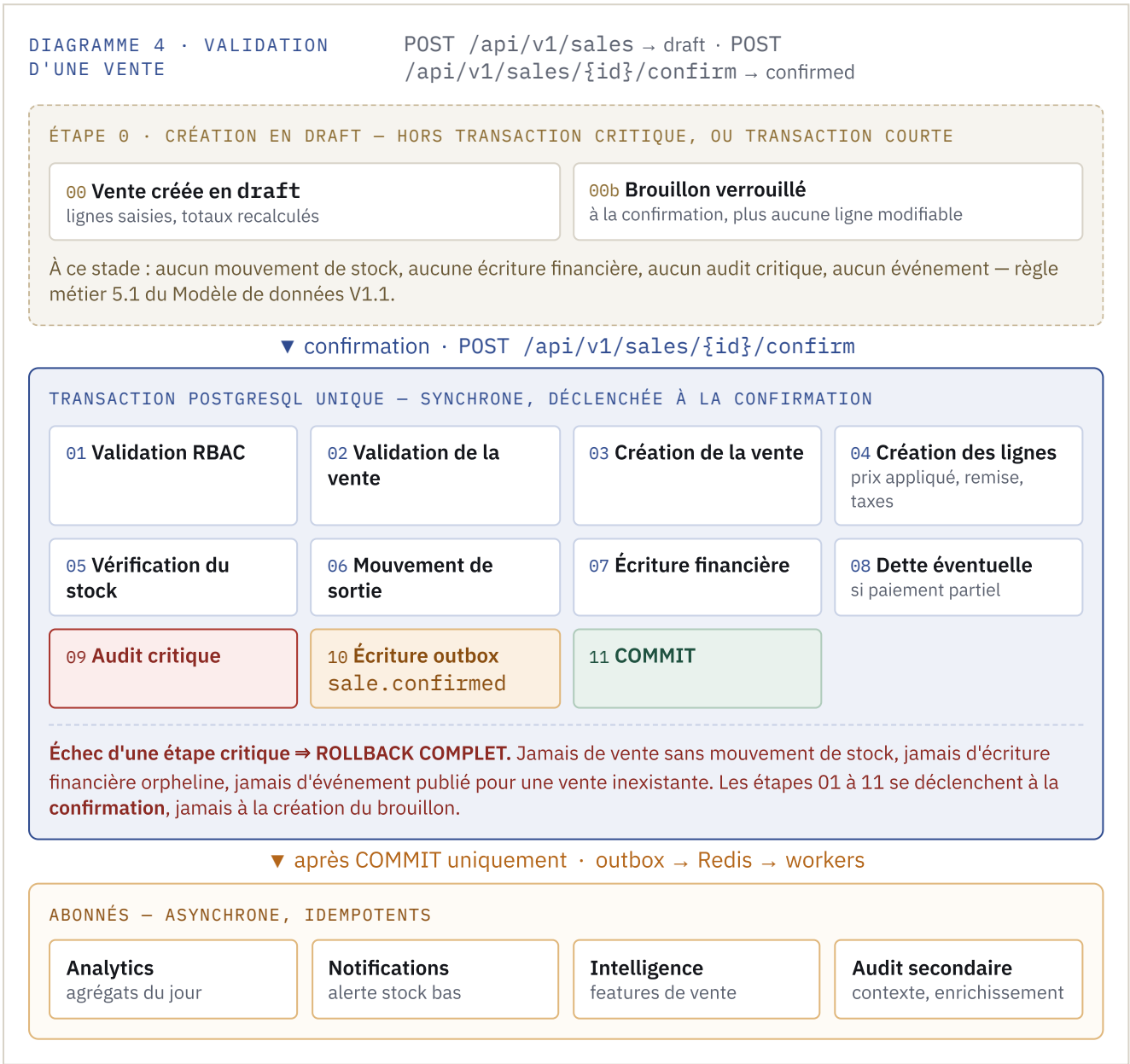
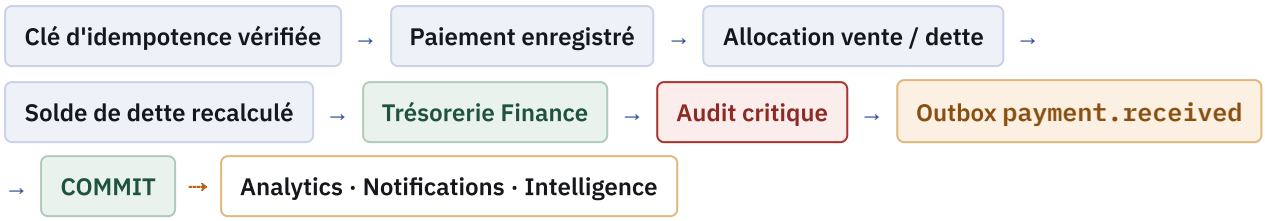
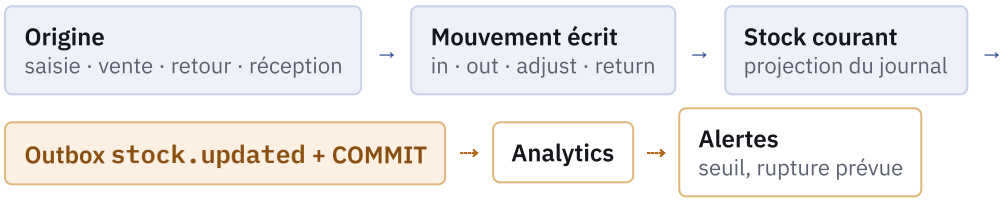


DIAGRAMME 5 · RÉCEPTION D'UN PAIEMENT — POST /api/v1/payments



Contrôles : montant alloué ≤ solde restant ; dette soldée ⇒ statut `settled` et `debt.settled` ; sur-paiement refusé par défaut. Une modification importante d'un paiement est journalisée en audit critique, avec valeur avant / après.

DIAGRAMME 6 · MOUVEMENT DE STOCK — POST /api/v1/inventory/movements



Le stock n'est jamais écrit en valeur absolue par un autre module : il est la somme des mouvements. Tout écart est auditable, tout historique rejouable pour l'Intelligence. Un ajustement manuel est une opération à auditer en critique.

AUDIT LOG — AUDIT CRITIQUE ET AUDIT ASYNCHRONE

L'audit essentiel ne doit pas dépendre de la disponibilité de Redis ou d'un worker. Deux canaux sont donc distingués, avec des garanties différentes.

CANAL 1 · AUDIT CRITIQUE

Dans la transaction métier

```
BEGIN
  opération métier
  INSERT audit_log (critique)
  INSERT outbox
COMMIT
```

- Écrit par le service, pas par un worker
- Perdu si et seulement si l'opération elle-même est annulée
- Append-only, aucune suppression applicative
- Contenu : acteur, entreprise, action, entité, valeur avant / après, horodatage, IP, origine

CANAL 2 · AUDIT SECONDAIRE

Après commit, par événement

```
COMMIT
→ outbox
→ Redis
→ worker
→ audit secondaire / traces
```

- Consultations, exports, connexions, enrichissement de contexte
- Tolère un retard ou une reprise
- Consommateur idempotent, dédoublonné par identifiant d'événement
- Ne porte jamais seul la preuve d'une opération critique

Figure 7 — Deux canaux d'audit. L'approche asynchrone reste la règle pour tout ce qui ne requiert pas de garantie transactionnelle immédiate.

Opérations exigeant un audit critique transactionnel

OPÉRATION	MOTIF
Modification d'une donnée financière critique	Montant, taxe, remise ou écriture de trésorerie : la trace doit exister dès que la donnée existe.
Suppression logique importante	Vente, paiement, produit, client : l'acteur et l'état antérieur doivent être récupérables.
Changement de permission ou de rôle	Élévation de privilège : condition d'enquête sur tout incident ultérieur.
Modification de paramètres sensibles	Devise, TVA, seuils, politique de sur-paiement : change l'interprétation de toutes les données suivantes.
Création ou modification d'une dette	Engagement financier vis-à-vis d'un tiers.
Modification importante d'un paiement	Réallocation, annulation, correction de montant.
Annulation ou retour de vente	Impacte simultanément stock, finance et dette.
Ajustement manuel de stock	Seule écriture de stock non issue d'un flux métier.

Tout le reste — consultation d'écran, export, connexion réussie, notification envoyée — relève du canal asynchrone.

PÉRIMÈTRES FONCTIONNELS V1

Module Sales — périmètre réduit

OBLIGATOIRE EN V1

- Création d'une vente
- Lignes de vente
- Prix appliqué (par ligne, traçable)
- Remise
- Taxes si activées pour l'entreprise
- Paiement total ou partiel
- Annulation
- Retour, lorsqu'il est nécessaire à la cohérence du stock
- Facture / reçu simple associé à la vente

REPORTÉ AUX VERSIONS FUTURES

- Devis avancés
- Gestion complète des commandes
- Workflow commercial avancé (étapes, validations, relances)
- Avoirs complexes
- Cycle commercial complet

Préparation sans implémentation : la vente porte un `statut` et un `type de document` extensibles, et les lignes sont déjà découplées de la vente — un devis futur sera un document amont, pas une refonte.

Module Finance — gestion, pas comptabilité générale

CoGes V1 n'est pas un logiciel de comptabilité générale.

Finance V1 répond à une question de pilotage — « où en est mon argent ? » — et non à une obligation comptable.

COUVERT EN V1

- Revenus issus des ventes
- Dépenses et charges
- Paiements
- Dettes
- Encaissements et décaissements
- Trésorerie simple
- Marge estimée
- Indicateurs financiers de gestion

HORS V1 — ÉVOLUTIONS FUTURES

- Comptabilité générale complète
- Plan comptable complet
- Grand livre comptable complet
- Rapprochement bancaire avancé
- Déclarations fiscales automatisées
- Comptabilité multi-normes

Les écritures V1 restent horodatées, typées et rattachées à leur pièce d'origine : un export comptable ultérieur reste possible sans reprise de données.

DIAGRAMME 7 — PIPELINE COGES INTELLIGENCE

L'Intelligence lit les données de gestion, ne bloque jamais une opération, et peut être désactivée sans interrompre CoGes. Ses modèles sont versionnés et n'écrivent que dans leurs propres tables.

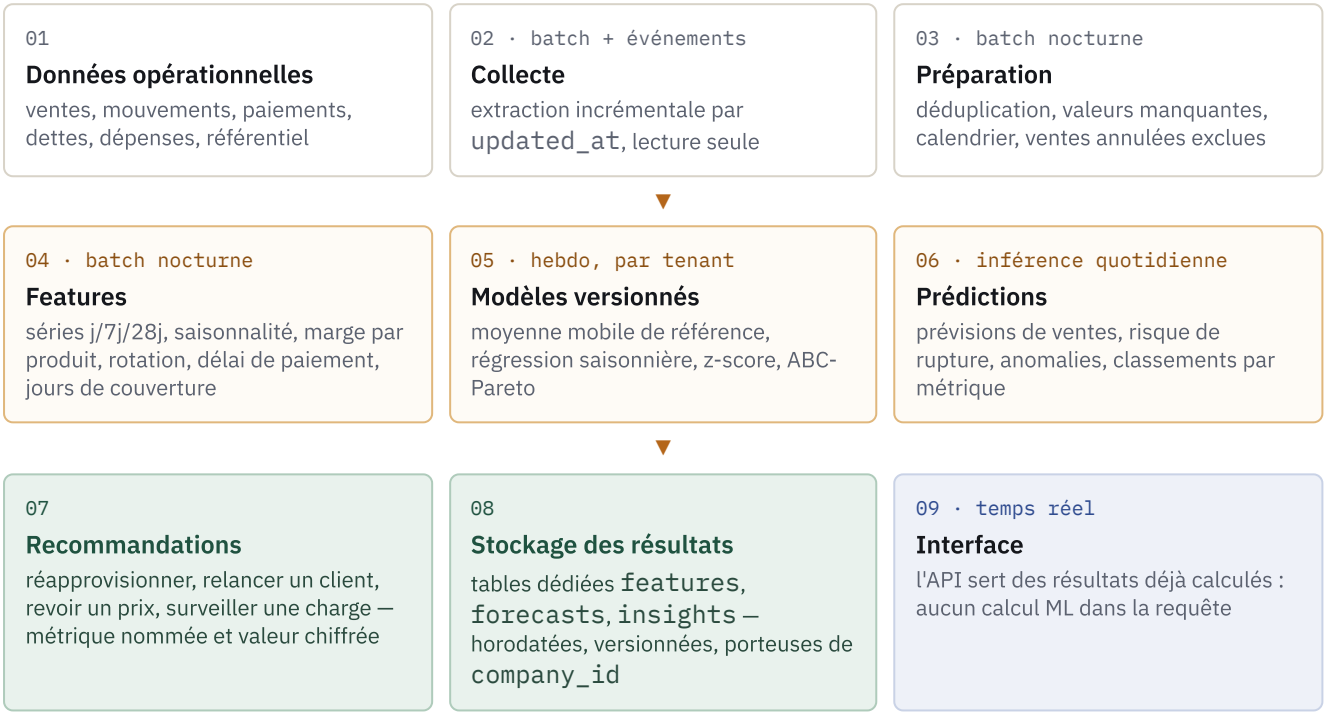


Figure 8 — Étapes 02 à 08 dans les conteneurs workers ; seule l'étape 09 est dans le chemin de la requête utilisateur.

« Valeur produit » : huit métriques distinctes, jamais équivalentes

Un « produit à forte valeur » n'existe pas comme métrique unique. Toute recommandation ou tout classement affiché indique explicitement la métrique sur laquelle il repose.

MÉTRIQUE	DÉFINITION	FORMULATION ATTENDUE À L'ÉCRAN
Chiffre d'affaires généré	Somme des ventes nettes sur la période.	« Produit A génère le plus fort chiffre d'affaires sur 30 jours. »
Marge générée	CA – coût d'achat, sur la période.	« Produit B génère la plus forte marge sur 30 jours. »
Quantité vendue	Unités écoulées.	« Produit C est le plus vendu en volume. »
Fréquence de vente	Nombre de ventes distinctes contenant le produit.	« Produit D apparaît dans le plus grand nombre de ventes. »
Rotation du stock	Quantité vendue rapportée au stock moyen détenu.	« Produit E a la plus forte rotation de stock. »
Vélocité des ventes	Unités vendues par jour sur une fenêtre glissante.	« Produit F s'écoule le plus rapidement, en unités par jour. »
Évolution de la demande	Variation de la vélocité entre deux fenêtres comparables.	« La demande du produit G progresse de 18 % par rapport au mois précédent. »
Potentiel futur estimé	Sortie d'un modèle de prévision, avec incertitude.	« Produit H est prévu en hausse — prévision du 18/09, modèle saisonnier v3, incertitude moyenne. »

CORRECT

« Produit A est actuellement le produit générant la plus forte marge. »
« Produit B est actuellement le produit ayant la plus forte rotation. »

INTERDIT

« Produit A est votre meilleur produit. »
Aucun classement sans métrique nommée, aucune fusion de plusieurs métriques en un score opaque non expliqué.

Règles d'affichage des prévisions

Date du calcul
Toute prévision affiche sa date de production, jamais « en direct ».

Modèle utilisé
Nom et version du modèle consultables depuis l'indicateur.

Niveau d'incertitude
Intervalle ou qualificatif lisible, dès que le modèle en produit un.

Données insuffisantes
Sous le seuil minimal d'historique : message explicite, jamais zéro ni valeur inventée.

Désactivation
Intelligence coupée ⇒ CoGes fonctionne, les blocs prédictifs disparaissent proprement.

DIAGRAMME 8 — SÉCURITÉ MULTI-TENANT : IMPLÉMENTATION DE RLS

Trois barrières indépendantes. Elles ne remplacent pas l'une l'autre : une erreur applicative ne suffit pas à provoquer une fuite, car la base refuse aussi.

ÉTAPE 1 · DÉTERMINATION DU TENANT

JWT vérifié → **Utilisateur chargé** →

company_id lu côté serveur
depuis le rattachement en base, jamais depuis un en-tête ni un paramètre client

→ **Contexte immuable**
(user, company_id, permissions)

▼

ÉTAPE 2 · TRANSMISSION À LA TRANSACTION

```
BEGIN;
  SET LOCAL app.company_id = '<uuid du tenant>';   -- portée = transaction
  -- toutes les requêtes du cas d'usage
COMMIT;                                            -- la variable expire avec la transaction
```

`SET LOCAL` et non `SET` : la valeur est liée à la transaction, elle disparaît au COMMIT ou au ROLLBACK. Aucune requête métier n'est exécutée hors transaction.

▼

ÉTAPE 3 · TROIS BARRIÈRES

Barrière 1 · Schéma
`company_id` NOT NULL sur toute table métier, contrainte `UNIQUE (company_id, id)` comme cible de référence, et FK **composites** `(company_id, child_id) → (company_id, parent_id)` sur chaque relation entre tables tenant-scoped ; `company_id` en tête de tout index composite.

Barrière 2 · Application
Repository de base qui injecte le filtre tenant ; un repository métier ne peut pas construire une requête sans lui, ni écrire une relation dont le `company_id` diffère du contexte — les FK composites rendent ce croisement impossible même en cas d'erreur applicative.

Barrière 3 · PostgreSQL
RLS activée et `FORCE`, politique comparant `company_id` à la variable de session ; le rôle applicatif n'est pas `BYPASSRLS`.

```
ALTER TABLE products ADD CONSTRAINT products_tenant_key UNIQUE (company_id, id);
ALTER TABLE sale_items
  ADD CONSTRAINT sale_items_product_fk
  FOREIGN KEY (company_id, product_id) REFERENCES products (company_id, id);

ALTER TABLE sales ENABLE ROW LEVEL SECURITY;
ALTER TABLE sales FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON sales
  USING      (company_id = current_setting('app.company_id', true)::uuid)
  WITH CHECK (company_id = current_setting('app.company_id', true)::uuid);
```

`WITH CHECK` autant que `USING` : la RLS protège aussi les écritures, pas seulement les lectures. Si la variable est absente, `current_setting(..., true)` renvoie NULL et la politique ne laisse rien passer — l'absence de contexte échoue de façon sûre.

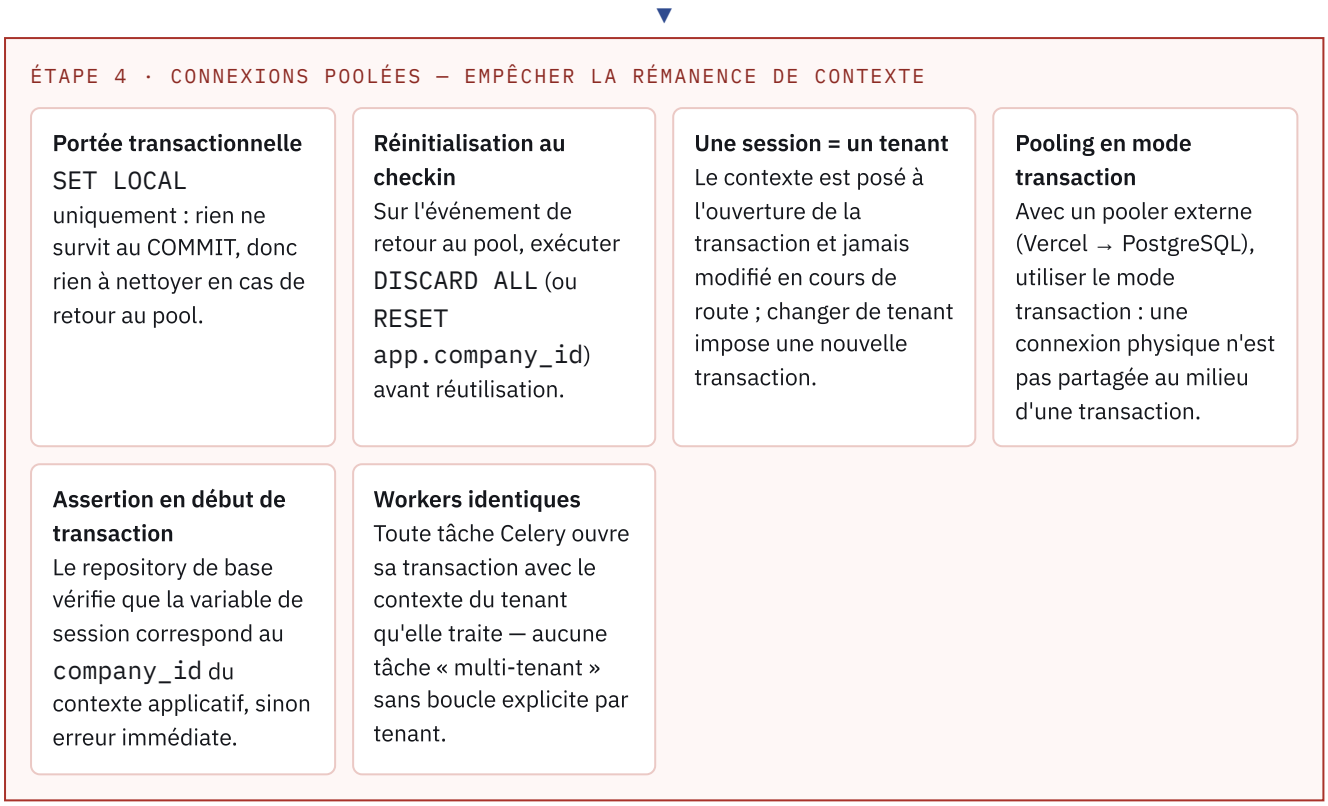


Figure 9 — Chaîne d'isolation, de l'authentification à la politique PostgreSQL, en incluant le comportement sur pool de connexions.

Tests automatisés de fuite inter-tenant

Suite obligatoire en intégration continue. Elle échoue ⇒ le déploiement est bloqué. Deux entreprises A et B sont créées avec des données propres, puis chaque endpoint est appelé en croisé.

CAS	ATTENDU	VÉRIFICATION
Entreprise A → donnée A	✅ **200**	Lecture, mise à jour et suppression autorisées sur ses propres enregistrements.
Entreprise A → donnée B	❌ **404**	Aucune donnée retournée, et un code indiscernable d'une ressource inexistante (pas de 403 qui révélerait l'existence).
Entreprise B → donnée A	❌ **404**	Symétrie vérifiée dans les deux sens pour chaque module.
Écriture croisée	❌ **refus**	`WITH CHECK` rejette l'insertion ou la mise à jour portant un `company_id` étranger.
Requête sans contexte	❌ **0 ligne**	Variable de session absente ⇒ aucune ligne visible : l'oubli de contexte ne produit jamais un accès global.
Réutilisation de connexion	❌ **étanche**	Enchaînement A puis B sur la même connexion physique : la seconde requête ne voit jamais les données de A.
Agrégats et exports	❌ **étanche**	Vues matérialisées, tableaux de bord et exports contrôlés au même titre que les endpoints CRUD.
Résultats d'Intelligence	❌ **étanche**	Tables `features`, `forecasts`, `insights` également porteuses de `company_id` et couvertes par RLS.

Un test générique parcourt la table des routes : tout nouvel endpoint est automatiquement soumis aux cas croisés, sans intervention du développeur.

PERFORMANCE — CHEMIN SYNCHRONE ET CHEMIN ASYNCHRONE

CHEMIN SYNCHRONE

Strictement ce qui est nécessaire pour terminer l'opération utilisateur

- Validation, RBAC, contexte tenant
- Écritures métier de la transaction
- Audit critique
- Écriture outbox
- Lecture d'agrégats déjà calculés

Cible : < 300 ms pour une écriture, < 200 ms pour une lecture de tableau de bord.

CHEMIN ASYNCHRONE

Tout ce qui peut être traité après le COMMIT

- Recalcul d'agrégats et vues matérialisées
- Notifications et alertes
- Audit secondaire
- Features, entraînement, inférence
- Exports volumineux, réconciliation nocturne

Aucun calcul ML lourd dans une requête utilisateur — sans exception.

Où s'exécute quoi

TRAITEMENT	EXÉCUTION	CADENCE & GARDE-FOUS
Écriture métier (vente, paiement, stock)	**Requête HTTP · Vercel**	Synchrone, transactionnel, audit critique inclus
Agrégats du tableau de bord	**Worker + cache**	Rafraîchis sur événement ; vue matérialisée reconstruite périodiquement
Notifications et alertes	**Worker**	À l'événement, déduplication par fenêtre
Préparation et features	**Batch nocturne**	1×/nuit par tenant, fenêtre glissante, reprise sur échec
Entraînement des modèles	**Batch hebdomadaire**	Par tenant, seuil minimal d'historique, artefact versionné
Inférence	**Batch quotidien**	Résultats persistés ; l'API ne fait que lire
Exports et documents	**Worker**	À la demande, fichier mis à disposition en fin de tâche

Index de référence : `(company_id, date)` en tête sur toute table à forte volumétrie — ventes, mouvements, paiements, écritures financières — et `(company_id, product_id, date)` pour les séries produit. Cache court par tenant sur les indicateurs d'accueil, invalidé par événement.

DIAGRAMME 9 — SÉPARATION V1 / VERSIONS FUTURES

OBLIGATOIRE EN V1

Core

- **Companies / Tenants**
- **Authentication**
- **Users**
- **Roles & Permissions**
- **Products**
- **Categories**
- **Customers**
- **Suppliers**
- **Sales** (réduit)
- **Inventory**
- **Finance** (gestion)
- **Payments**
- **Debts**
- **Expenses**

Management

- **Dashboard**
- **Analytics essentielles**
- **Notifications principales**
- **Audit Log**
- **Import/export CSV · Excel** si nécessaire

Intelligence

- **Statistiques descriptives**
- **Tendances**
- **Anomalies simples**
- **Alertes de stock**
- **Prévisions simples** si historique suffisant
- **Risque de rupture**
- **Produits par métrique**
- **Recommandations explicables**

VERSIONS FUTURES — NON IMPLÉMENTÉES, RESTÉES POSSIBLES

Fonctionnel

- Achats avancés
- Commandes fournisseurs
- Multi-entrepôts
- CRM avancé
- RH
- Paie
- Comptabilité avancée
- Facturation électronique
- Abonnements
- Automatisations avancées

Canaux

- Application mobile
- Mode hors connexion

Technique

- Modèles ML avancés
- Architecture distribuée / microservices
- Kafka / RabbitMQ
- Entrepôt de données dédié
- Agents IA conversationnels avancés

Ce que l'architecture V1 garantit pour eux : frontières de modules explicites (extraction possible), outbox (remplacement du transport sans toucher aux émetteurs), Intelligence en lecture seule (isolable), stock en journal de mouvements (multi-entrepôts additif), écritures financières typées et rattachées à leur pièce (export comptable ultérieur).

DIAGRAMME 10 — ARBORESCENCE FINALE DU PROJET

Backend — FastAPI (Vercel) + workers (Docker)

```
backend/
  app/
    main.py                 # entrée API (Vercel)
    core/
      config.py  security.py
      database.py           # engine, pool, checkin reset
      tenancy.py            # contexte + SET LOCAL
      deps.py               # auth, RBAC, tenant
      audit.py              # audit critique transactionnel
      events/
        types.py  outbox.py  bus.py
      errors.py  logging.py
    modules/
      companies/
      auth/
      users/
      rbac/
      products/
      categories/
      customers/
      suppliers/
      sales/                # V1 réduit
        router.py
        service.py
        repository.py
        models.py
        schemas.py
        events.py
      inventory/
      finance/
      payments/
      debts/
      expenses/
      dashboard/
      analytics/
      notifications/
      audit/
      importexport/
      intelligence/
        features/
        models/
        pipelines/
        serving.py          # lecture des résultats
    workers/                # image Docker
      celery_app.py
      beat.py               # planification
      tasks/
        outbox_relay.py
        analytics.py
        notifications.py
        ml_features.py
        ml_train.py
        ml_infer.py
        exports.py
```

```
        reconcile.py
    migrations/             # Alembic
  tests/
    unit/  integration/
    tenancy/                # fuite inter-tenant
    architecture/           # imports inter-modules
  Dockerfile.worker
  vercel.json
  pyproject.toml
```

Frontend — Next.js (Vercel)

```
frontend/
  app/
    (auth)/login/
    (app)/
      dashboard/
      products/
      categories/
      customers/
      suppliers/
      sales/
        new/                # vente + paiement
        [id]/               # reçu simple
      inventory/
        movements/
      finance/
        expenses/
        debts/
        payments/
      analytics/
      insights/             # CoGes Intelligence
      settings/
        users/  roles/
        company/            # devise, TVA, seuils
        audit/
    layout.tsx
  components/
    ui/                     # primitives
    charts/
    data-table/
    alerts/
    forecast/               # date, modèle, incertitude
  lib/
    api/                    # client REST typé
    auth/
    permissions.ts
    format/                 # devise décimale, dates
  hooks/
  types/                    # générés depuis OpenAPI
  package.json

deploy/
  docker-compose.worker.yml
  README-infra.md           # base, cache, scheduler
```

Un module = un dossier, même découpage interne partout, aucune logique métier hors `modules/`. `tests/architecture/` échoue si un module importe le repository d'un autre ; `tests/tenancy/` échoue sur la moindre fuite. Côté frontend, `types/` est généré depuis OpenAPI pour que le contrat ne dérive pas.

ANNEXE A — SOURCES MERMAID DES DIX DIAGRAMMES

Équivalents textuels à versionner dans le dépôt de documentation, à côté du code.

A.1 — Architecture logique globale

```
flowchart TB
  U[Utilisateur] --> FE[Next.js · Vercel]
  FE --> API[API FastAPI · routers, Pydantic, auth, tenant resolver]
  API --> SVC[Services metier du monolithe]
  SVC --> RP[Repositories · filtre company_id]
  RP --> DB[(PostgreSQL manage · RLS)]
  SVC --> AUD[Audit critique · meme transaction]
  AUD --> DB
  SVC --> OBX[(Table outbox)]
  OBX --> DB
  OBX -. apres commit .-> RDS[(Redis manage)]
  RDS --> WK[Workers Celery · Docker]
  WK --> ANA[Analytics]
  WK --> NOT[Notifications]
  WK --> AUD2[Audit secondaire]
  WK --> ML[CoGes Intelligence]
  ML -. lecture seule .-> DB
  WK --> DB
```

A.2 — Déploiement Vercel + infrastructure externe

```
flowchart LR
  subgraph V[Vercel · chemin utilisateur]
    NX[Application Next.js]
    AP[API FastAPI · requetes courtes]
    CD[Deploiement continu depuis Git]
    TLS[HTTPS · domaine · edge]
  end
  subgraph X[Infrastructure externe · runtime controle]
    PG[(PostgreSQL manage + pooler)]
    RD[(Redis manage)]
    CW[Workers Celery · Docker]
    MLW[Taches ML et batch · Docker]
    SCH[Scheduler]
    EXP[Exports volumineux]
  end
  NX --> AP
  AP --> PG
  AP --> RD
  RD --> CW
  SCH --> MLW
  CW --> PG
  MLW --> PG
  MLW --> EXP
  CW -.->|aucun appel entrant| V
```

A.3 — Modules backend

```
flowchart TB
  subgraph E[Bande E · Restitution]
    DSH[Dashboard]; ANA[Analytics]; NOT[Notifications]; INT[CoGes Intelligence]
  end
  subgraph D[Bande D · Finance de gestion]
    FIN[Finance]; PAY[Payments]; DEB[Debts]; EXP[Expenses]
  end
  subgraph C[Bande C · Operations]
    SAL[Sales · V1 reduit]; INV[Inventory]
  end
  subgraph B[Bande B · Referentiel]
    PRD[Products]; CAT[Categories]; CUS[Customers]; SUP[Suppliers]
  end
  subgraph A[Bande A · Plateforme]
    COM[Companies]; AUT[Auth]; USR[Users]; RBA[Roles and Permissions]
    AUD[Audit Log]; BUS[(Outbox et bus)]; TEN[Tenant context]
  end
  SAL --> PRD; SAL --> CUS; SAL --> INV; SAL --> FIN; SAL --> AUD
  INV --> PRD; INV --> SUP
  PAY --> DEB; PAY --> FIN; PAY --> AUD
  DEB --> CUS; DEB --> SUP; DEB --> FIN; DEB --> AUD
  EXP --> FIN
  DSH --> ANA; DSH --> NOT; DSH --> INT
  ANA -. lecture .-> SAL; ANA -. lecture .-> INV; ANA -. lecture .-> FIN
  INT -. lecture .-> ANA
  SAL --> BUS; INV --> BUS; PAY --> BUS; DEB --> BUS; EXP --> BUS
  BUS --> NOT; BUS --> ANA; BUS --> INT; BUS --> AUD
```

A.4 — Flux d'une vente

```
sequenceDiagram
  participant U as Utilisateur
  participant API as API FastAPI
  participant SAL as Sales
  participant INV as Inventory
  participant FIN as Finance
  participant DEB as Debts
  participant AUD as Audit critique
  participant OBX as Outbox
  participant WK as Workers
  U->>API: POST /api/v1/sales
  API->>SAL: 00 creation en draft · brouillon verrouillable
  SAL-->>U: 201 vente en draft
  U->>API: POST /api/v1/sales/{id}/confirm
  API->>SAL: confirm_sale(id, ctx)
  Note over SAL,OBX: BEGIN · SET LOCAL app.company_id
  SAL->>SAL: 01 RBAC · 02 validation
  SAL->>SAL: 03 vente · 04 lignes
  SAL->>INV: 05 verification stock · 06 mouvement de sortie
  SAL->>FIN: 07 ecriture financiere
  SAL->>DEB: 08 dette si paiement partiel
  SAL->>AUD: 09 audit critique
  SAL->>OBX: 10 sale.confirmed
  Note over SAL,OBX: 11 COMMIT · sinon ROLLBACK COMPLET
  SAL-->>API: vente confirmee
  API-->>U: 200 OK · statut confirmed
  OBX->>WK: relais via Redis
  WK->>WK: Analytics · Notifications · Intelligence · Audit secondaire
```

A.5 — Flux d'un paiement

```
flowchart TB
  IN[POST /api/v1/payments] --> IDEM{Cle d idempotence deja vue ?}
  IDEM -- oui --> RET[Retour du resultat initial]
  IDEM -- non --> TX[BEGIN transaction]
  TX --> P1[Paiement enregistre]
  P1 --> P2[Allocation vente ou dette]
  P2 --> CHK{Montant alloue <= solde restant ?}
  CHK -- non --> RB[ROLLBACK · 422]
  CHK -- oui --> P3[Solde de dette recalcule]
  P3 --> P4{Solde = 0 ?}
  P4 -- oui --> P5[Statut settled · debt.settled]
  P4 -- non --> P6[Dette partiellement soldee]
  P5 --> FINX[Tresorerie Finance]
  P6 --> FINX
  FINX --> AUDX[Audit critique]
  AUDX --> OBXX[Outbox payment.received]
  OBXX --> CM[COMMIT]
  CM -. apres commit .-> WKX[Analytics · Notifications · Intelligence]
```

A.6 — Flux d'un mouvement de stock

```
flowchart LR
  SRC{Origine du mouvement}
  SRC -->|vente| MV[Mouvement out]
  SRC -->|retour| MV2[Mouvement return]
  SRC -->|reception| MV3[Mouvement in]
  SRC -->|ajustement manuel| MV4[Mouvement adjust + audit critique]
  MV --> LEDGER[(Journal des mouvements)]
  MV2 --> LEDGER
  MV3 --> LEDGER
  MV4 --> LEDGER
  LEDGER --> CUR[Stock courant · projection]
  CUR --> OBX[Outbox stock.updated]
  OBX -. apres commit .-> ANA[Analytics]
  OBX -. apres commit .-> ALR[Alertes seuil et rupture prevue]
  ALR --> INT[CoGes Intelligence · risque de rupture]
```

A.7 — Pipeline CoGes Intelligence

```
flowchart LR
  OPS[(Donnees operationnelles)] --> COL[02 Collecte incrementale]
  COL --> CLN[03 Preparation]
  CLN --> FE[04 Features]
  FE --> TR[05 Entrainement hebdomadaire par tenant]
  TR --> MOD[Modeles versionnes]
  MOD --> INF[06 Inference quotidienne]
  INF --> OUT[Previsions · anomalies · risque de rupture · classements par metrique]
  OUT --> REC[07 Recommandations avec metrique nommee]
  REC --> ST[(08 Tables features / forecasts / insights)]
  ST --> UI[09 Dashboard et ecran Insights]
  MOD -. jamais d ecriture transactionnelle .-> OPS
  UI -->|si historique insuffisant| MSG[Message donnees insuffisantes]
```

A.8 — Sécurité multi-tenant

```
flowchart TB
  JWT[JWT verifie] --> USR[Utilisateur charge en base]
  USR --> CID[company_id determine cote serveur]
  CID --> CTX[Contexte immuable user + company_id + permissions]
  CTX --> TX[BEGIN · SET LOCAL app.company_id]
  TX --> B1[Barriere 1 · company_id NOT NULL · UNIQUE company_id,id · FK composites]
  TX --> B2[Barriere 2 · repository de base · filtre impose · pas de croisement tenant]
  TX --> B3[Barriere 3 · RLS FORCE · USING et WITH CHECK]
  B1 --> DB[(PostgreSQL)]
  B2 --> DB
  B3 --> DB
  TX --> CM[COMMIT · la variable expire]
  CM --> POOL[Retour au pool · DISCARD ALL au checkin]
  POOL --> NEXT[Transaction suivante · contexte repose a zero]
  DB --> TEST[Tests CI de fuite inter-tenant]
  TEST --> T1[A vers donnee A : autorise]
  TEST --> T2[A vers donnee B : refuse]
  TEST --> T3[B vers donnee A : refuse]
  TEST --> T4[Sans contexte : aucune ligne]
  TEST --> T5[Connexion reutilisee : etanche]
```

A.9 — Séparation V1 / versions futures

```
flowchart LR
  subgraph V1[OBLIGATOIRE EN V1]
    direction TB
    CORE[Core · Companies, Auth, Users, RBAC, Products, Categories,
          Customers, Suppliers, Sales reduit, Inventory,
          Finance de gestion, Payments, Debts, Expenses]
    MGT[Management · Dashboard, Analytics essentielles,
         Notifications principales, Audit Log, Import/export CSV]
    INTL[Intelligence · descriptif, tendances, anomalies simples,
          alertes stock, previsions simples, risque de rupture,
          produits par metrique, recommandations explicables]
  end
  subgraph FUT[VERSIONS FUTURES]
    direction TB
    F1[Achats avances · commandes fournisseurs · multi-entrepots]
    F2[CRM avance · RH · paie · abonnements · automatisations]
    F3[Comptabilite avancee · facturation electronique]
    F4[Mobile · mode hors connexion]
    F5[ML avance · microservices · Kafka/RabbitMQ ·
       entrepot dedie · agents IA conversationnels]
  end
  CORE --> FUT
  MGT --> FUT
  INTL --> FUT
  V1 -.->|frontieres de modules, outbox, lecture seule ML,
  journal de mouvements| FUT
```

A.10 — Arborescence du projet

```
flowchart TB
  ROOT[coges/] --> BE[backend/]
  ROOT --> FEX[frontend/]
  ROOT --> DEP[deploy/]
  BE --> APPX[app/]
  APPX --> CORE2[core/ · config, database, tenancy, audit, events]
  APPX --> MODS[modules/ · un dossier par module]
  APPX --> WKS[workers/ · celery_app, beat, tasks]
  APPX --> MIG[migrations/ · Alembic]
  MODS --> SHAPE[router · service · repository · models · schemas · events]
  BE --> TST[tests/ · unit, integration, tenancy, architecture]
  BE --> DKR[Dockerfile.worker + vercel.json]
  FEX --> APPR[app/ · routes App Router]
  FEX --> CMP[components/ · ui, charts, data-table, alerts, forecast]
  FEX --> LIBX[lib/ · api, auth, permissions, format]
  FEX --> TYP[types/ · generes depuis OpenAPI]
  DEP --> INFRA[docker-compose.worker.yml + README-infra]
```

DÉCISIONS DÉFINITIVEMENT RETENUES POUR COGES V1

Liste de référence pour le développement. Rien au-delà de cette liste n'est à construire en V1.

01	**Monolithe modulaire FastAPI, un module par dossier** Façade de services, modèles privés, aucun import du repository d'un autre module — vérifié par un test d'architecture.
02	**Chemin utilisateur sur Vercel, traitements longs en Docker externe** Next.js et API sur Vercel, déployés depuis Git ; PostgreSQL et Redis managés ; workers, ML, batch et exports dans des conteneurs.
03	**Une transaction PostgreSQL par cas d'usage, rollback complet** Vente : RBAC, validation, vente, lignes, stock, mouvement, écriture financière, dette, audit critique, outbox, COMMIT. Échec d'une étape critique ⇒ ROLLBACK COMPLET.
04	**Outbox transactionnelle, publication après commit uniquement** L'événement est écrit avec la donnée ; un relais l'envoie dans Redis. Consommateurs idempotents, reprises avec back-off, file de rebut surveillée.
05	**Audit à deux canaux** Audit critique dans la transaction pour les opérations financières, les suppressions importantes, les changements de permission, les paramètres sensibles, les dettes, les paiements, les annulations et les ajustements de stock. Audit secondaire par événement pour le reste. Append-only.
06	**Isolation multi-tenant à trois barrières** `company_id` NOT NULL indexé, filtre imposé par le repository de base, RLS `FORCE` avec `USING` et `WITH CHECK`. Contexte posé par `SET LOCAL` en début de transaction, réinitialisé au retour au pool, pooling en mode transaction.
07	**Tests de fuite inter-tenant bloquants en CI** A → A autorisé, A → B refusé, B → A refusé, écriture croisée refusée, requête sans contexte sans résultat, connexion réutilisée étanche. Test générique appliqué à toute nouvelle route.
08	**Sales V1 réduit** Vente, lignes, prix appliqué, remise, taxes si activées, paiement total ou partiel, annulation, retour, reçu simple. Pas de devis avancés, de commandes, de workflow commercial ni d'avoirs complexes.
09	**Finance de gestion, pas de comptabilité générale** Revenus, dépenses, paiements, dettes, encaissements, décaissements, trésorerie simple, marge estimée, indicateurs. Aucun plan comptable, grand livre, rapprochement avancé, déclaration fiscale ni multi-normes.
10	**Stock en journal de mouvements** Le stock courant est une projection ; aucune écriture en valeur absolue par un autre module. Historique auditable et rejouable.
11	**Intelligence hors du transactionnel, en lecture seule** Collecte, préparation, features, modèles versionnés, inférence, recommandations, résultats dans des tables dédiées. Désactivable sans interrompre CoGes. Aucun calcul ML dans une requête utilisateur.

12 **Valeur produit = métrique nommée**
CA, marge, quantité, fréquence, rotation, vélocité, évolution de la demande, potentiel estimé — jamais présentés comme équivalents. Toute recommandation nomme sa métrique et affiche sa valeur.

13 **Prévisions toujours qualifiées**
Date de calcul, modèle et version, incertitude quand elle existe, message « données insuffisantes » sous le seuil minimal d'historique.

14 **Performance : agrégats pré-calculés et index disciplinés**
Vues matérialisées, cache court par tenant invalidé par événement, index `(company_id, date)` et `(company_id, product_id, date)`, pagination obligatoire.

15 **Montants en décimal, de bout en bout**
`NUMERIC` en base, `Decimal` en Python, formatage devise côté frontend. Aucun flottant sur une écriture financière.

16 **Sécurité applicative de base**
JWT courts + refresh rotatif, Argon2, RBAC vérifié côté serveur, schémas Pydantic stricts, limitation de débit par IP et par tenant, clés d'idempotence sur les écritures financières, secrets hors dépôt, TLS, sauvegardes chiffrées avec PITR et restauration testée.

17 **Migrations Alembic obligatoires et réversibles**
Exécutées comme étape de déploiement, jamais au démarrage d'une requête ; déploiement en deux temps pour tout changement destructif.

18 **Tout le reste est hors V1**
Achats avancés, commandes fournisseurs, multi-entrepôts, CRM, RH, paie, comptabilité avancée, facturation électronique, mobile, hors connexion, abonnements, automatisations, ML avancé, microservices, Kafka/RabbitMQ, entrepôt dédié, agents conversationnels. Non implémentés, non préparés au-delà des frontières déjà posées.

Risques techniques & réduction

RISQUE	GRAVITÉ	RÉDUCTION
Fuite de données entre tenants	**Critique**	Trois barrières + suite de tests croisés bloquante en CI + réinitialisation du contexte au retour au pool.
Contexte de tenant rémanent sur connexion poolée	**Critique**	`SET LOCAL` uniquement, `DISCARD ALL` au checkin, pooling en mode transaction, assertion en début de transaction.
Perte de trace d'une opération sensible	**Élevée**	Audit critique dans la transaction : indépendant de Redis et des workers.
Dérive du monolithe	**Élevée**	Test d'architecture interdisant les imports transversaux ; revue de toute nouvelle dépendance inter-modules.
Incohérence ventes / stock / finance	**Élevée**	Une transaction par cas d'usage, contraintes en base, réconciliation nocturne qui remonte les écarts.
Traitement long lancé dans une requête	**Élevée**	Règle de zone : tout traitement long vit dans un conteneur worker ; budget de temps surveillé côté API.
Épuisement des connexions base depuis Vercel	**Moyenne**	Pooler externe en mode transaction, taille de pool bornée, pas de connexion persistante par requête.
IA peu fiable sur faible historique	**Moyenne**	Seuil minimal par modèle, incertitude affichée, message « données insuffisantes ».
Tâches asynchrones en échec silencieux	**Moyenne**	Reprises avec back-off, file de rebut surveillée, alerte d'exploitation, consommateurs idempotents.
Migrations risquées en production	**Moyenne**	Migrations réversibles, déploiement en deux temps, sauvegarde préalable.